coursera

Hands-On Learning : Enterprise RAG Implementation: Production Q&A System with Sub-Second Response

Description

Design and implement production-grade RAG system with semantic search, multi-tier caching for 60% cost reduction, comprehensive monitoring with Prometheus/Grafana, and auto-scaling infrastructure handling 10x traffic spikes while meeting strict enterprise SLAs.

Learning Objectives

By the end of this activity, learners will be able to:

- Implement RAG architecture with vector databases and semantic retrieval for production scale
- Design multi-tier caching strategies achieving cost reduction while maintaining data freshness
- Configure comprehensive monitoring with leading indicators for proactive issue detection
- Implement auto-scaling infrastructure with Kubernetes handling variable traffic patterns

Assignment Components

Introductory Slide:

- **Title:** Solutions Architect Mission: Launch GlobalAI's Enterprise Q&A for 50,000 Users
- **Subtitle:** 72 hours to deploy production-grade RAG system with 99.9% uptime SLA or lose Fortune 100 client

Tools to be used:

- LangChain (RAG framework)
- Pinecone or Weaviate (vector database)
- Redis (caching layer)
- Prometheus & Grafana (monitoring)
- Kubernetes (orchestration and auto-scaling)
- Apache JMeter (load testing)

Scenario

You're the Solutions Architect at GlobalAI, hired to deploy a production Q&A system for TechCorp—a Fortune 100 client with 50,000 employees launching Monday. The prototype works with 10 test users but has never faced production load. TechCorp's requirements are uncompromising: sub-2-second response for 95th percentile queries, 99.9% uptime SLA ($100,000 penalties per hour of downtime), costs under $50,000 monthly, handle 10x traffic spikes during business hours, enterprise security and audit compliance.

You must implement complete RAG system with semantic search across 500GB knowledge base, design hierarchical caching (in-memory, Redis, CDN) balancing cost and freshness, set up monitoring with leading indicators (latency percentiles, cache hit rates, token consumption), and create Kubernetes auto-scaling handling traffic variations. The CEO told TechCorp's CIO deployment is guaranteed—canceling destroys GlobalAI's reputation and revenue.

Instructions for learners

Step 1: RAG Architecture Design and Implementation

- Set up vector database (Pinecone/Weaviate) and configure embeddings model
- Implement document chunking strategy balancing context preservation and retrieval granularity
- Build LangChain RAG pipeline with RetrievalQA chain connecting embeddings to LLM

- Configure semantic search with appropriate similarity threshold and top-k retrieval
- Test retrieval quality with sample queries and validate response accuracy

Step 2: Multi-Tier Caching Strategy Implementation

- Set up Redis caching layer with intelligent TTL configuration
- Implement cache key design using query fingerprinting for effective hit rates

- Configure cache invalidation strategy balancing freshness and cost reduction
- Add in-memory caching for frequently accessed queries (LRU cache)
- Measure cache hit rates and calculate cost savings from reduced API calls

Step 3: Monitoring and Observability Setup (4 minutes)

- Configure Prometheus to collect metrics: latency percentiles (p50, p95, p99), throughput, error rates
- Set up Grafana dashboards visualizing real-time system health and performance

- Implement custom metrics: cache hit rate, token consumption, cost tracking
- Create alerting rules for SLA violations and degradation warnings
- Document leading indicators that predict issues before user impact

Step 4: Auto-Scaling Configuration and Load Testing

- Configure Kubernetes Horizontal Pod Autoscaler (HPA) based on CPU and custom metrics
- Set up load balancer with health checks and traffic distribution policies
- Run Apache JMeter load tests simulating 10,000 concurrent users
- Validate sub-2-second response times at 95th percentile under load
- Test auto-scaling behavior during traffic spikes and measure response time

Step 5: Documentation and Deployment Planning

- Document architecture decisions with trade-off analysis (cost vs performance)
- Create operational runbook covering incident response and capacity planning
- Compile performance testing results proving SLA compliance
- Prepare cost optimization report showing 60% reduction strategies

Deliverable for Submission (AI Grading):

Deliverable 1: Complete RAG Implementation with Caching

Question Prompt: What RAG architecture (vector database, chunking strategy, retrieval approach) and multi-tier caching design (Redis + in-memory) did you implement to achieve sub-2-second latency for 50,000 concurrent users, and how does your approach balance cost reduction with data freshness requirements?

Response Type: File Upload

File Types: Coding files (.py, .yml, .json), Text (.md, .txt for documentation), Compressed archives (.zip)

Deliverable 2: Monitoring Dashboard and Auto-Scaling Configuration

Question Prompt: What monitoring metrics (latency percentiles, cache hit rates, token consumption) and auto-scaling strategies (Kubernetes HPA/VPA) did you implement to ensure 99.9% uptime, and how do your leading indicators enable proactive issue detection rather than reactive error response?

Response Type: File Upload

File Types: Coding files (.yml, .json), Image (.png, .jpg for dashboard screenshots), Text (.txt, .md)

Deliverable 3: Performance Testing Results and Cost Optimization Report

Question Prompt: What do your load testing results (concurrent users, latency percentiles, cache effectiveness) reveal about production readiness, and how did you achieve 60% cost reduction while maintaining sub-2-second response times and data freshness requirements?

Response Type: File Upload or Rich Text

File Types: Text (.txt, .pdf, .docx for reports), Image (.png, .jpg for test results), Video (.mp4, .mov - max 15 min for demonstrations)

Deliverable 4: Operational Runbook Documentation

Question Prompt: How would operations teams use your runbook to maintain system reliability (incident response procedures, capacity planning, troubleshooting playbook), and why are your documented architectural trade-offs (cost vs performance vs reliability) critical for adapting to changing business requirements?

Response Type: File Upload

File Types: Text (.txt, .pdf, .docx, .md), Audio (.mp3, .wav for verbal documentation), Video (.mp4, .mov - max 15 min)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.